

MIDTERM PROJECT PROGRESS REPORT • IT CAPSTONE

AI Meeting Assistant

A Privacy-Preserving, Edge-Native Meeting Intelligence System

Course: Emerging Topics in Information Technology

Architecture Track: Edge AI & Multi-Agent Modular Monolith

Academic Term: Spring Semester 2026

Submission Date: September 29, 2026

1.0 Project Identification & Team Roster

The project is developed by a collaborative team of four student engineers, organized into specialized subsystem ownership:

Student Full Name	Subsystem Role	Assigned Modules	Core Responsibilities
Lương Việt Anh <i>(Team Leader)</i>	Orchestration, Agent 2 (Summarization) & Core Algorithm	main.py mmr_extractor.py meeting-assistant-ui/	Pipeline design, MMR redundancy elimination algorithm, Llama 3 Map-Reduce engine, full React UI client.
Triệu Quang Thiện	Agent 1: Speech-to-Text (STT) Engineering	agent1_transcribe.py tests/test_stt.py	OpenAI Whisper integration, audio ingestion, FFmpeg runtime resolution, segment timestamp alignment.
Nguyễn Quang Minh	Agent 3: Action Items & Information Extraction	agent3_action_items.py	Constrained JSON prompt design, bracket-depth lexical scanning, trailing comma and quote recovery.
Đoàn Hoàng Long	Agent 4: Database & Data Persistence	database/models.py database/crud.py database/db.py	SQLite database layer, WAL concurrency mode, Pydantic v2 data contracts, historical retrieval APIs.

2.0 Executive Overview

In modern corporate and educational environments, meetings consume up to 30% of working hours, yet extracting actionable insights remains manual, error-prone, and inefficient. While commercial cloud solutions (e.g., Otter.ai, Microsoft Copilot) exist, they present severe **data privacy vulnerabilities** and significant recurring subscription costs. Transmitting internal financial deliberations, intellectual property discussions, and personnel reviews to cloud APIs violates enterprise security guidelines (e.g., GDPR, SOC 2).

AI Meeting Assistant solves this paradigm by delivering an enterprise-ready meeting transcription, summarization, and action item tracking platform engineered to run **100% offline on standard consumer hardware** (laptops with 8GB–16GB RAM, with or without dedicated NVIDIA GPUs). The system combines state-of-the-art automatic speech recognition (OpenAI Whisper) with a novel **Hybrid Extractive-Abstractive Summarization** approach: a high-speed vector-space algorithm (Maximal Marginal Relevance) pre-filters conversational noise before feeding high-density decision points into a local Large Language Model (Meta Llama 3) via a sliding-window Map-Reduce architecture.

3.0 Problems and Objectives

3.1 Problem Statement

1. **Confidentiality and Privacy Risks:** Enterprise and academic stakeholders cannot upload proprietary audio records to third-party cloud LLM endpoints without risking data leakage and regulatory non-compliance.
2. **Hardware and Memory Bottlenecks on Edge Devices:** Consumer laptops lack data-center VRAM (e.g., A100 GPUs). Running 8-billion parameter models directly over raw multi-hour transcripts triggers Out-Of-Memory (OOM) failures or intolerable latency (often > 15 minutes per file).
3. **Conversational Redundancy and Disfluency:** Human speech contains 40%–50% lexical noise, verbal fillers ("um", "uh", "you know"), greetings, and circular arguments. Injecting raw speech directly into an LLM exhausts the context window and triggers hallucination.
4. **System Brittle Failures during Live Operations:** Long-running synchronous AI calls can freeze backend event loops, causing browser network timeouts and presentation crashes when demoed on low-spec hardware.

3.2 Project Objectives

- **Zero-Cloud Guarantee:** Build a self-contained local desktop system with zero external API calls for audio transcription, text summarization, or structured storage.
- **Algorithmic Compression ($\geq 35\%$):** Implement a pure-Python Maximal Marginal Relevance (MMR) filter that mathematically eliminates repetitive speech without losing critical decision statements.
- **Scalable Map-Reduce Pipeline:** Design an overlapping sliding-window chunking strategy capable of processing transcripts of arbitrary length (1–3 hours) within a constrained 2048/4096 token budget.
- **Robust Data Contracts & Defensive Engineering:** Guarantee that malformed model outputs (e.g., invalid JSON syntax from quantized models) are sanitized automatically without crashing the user interface.
- **Production-Grade UX:** Deliver an interactive React dashboard with dual transcript switching (Raw vs. Filtered), live telemetry metrics, task completion checklists, and persistent SQLite history.

4.0 Technical Approaches & Trade-Off Analysis

4.1 Technology Stack & Architectural Selection

- **Speech-to-Text Engine:** OpenAI Whisper (tiny model for CPU-only laptops; base model for NVIDIA CUDA-accelerated systems). Audio is converted into Log-Mel spectrogram representations (80-channel filterbank) and processed via an encoder-decoder sequence-to-sequence Transformer.
- **Extractive Algorithmic Layer:** Pure-Python Vector Space Model employing **Sparse TF-IDF** and **Maximal Marginal Relevance (MMR)**. Evaluates relevance to a meeting centroid vector Q while penalizing cosine similarity against already selected sentences.
- **Abstractive Summarization Engine:** Meta Llama 3 (8B quantized or Llama 3.2 1B/3B) running on local Ollama, orchestrated through a sliding-window Map-Reduce pipeline.
- **Structured Task Extraction:** Constrained JSON prompt format paired with a custom Lexical Bracket-Depth Scanner and single-to-double quote recovery filter.
- **Persistence Layer:** Embedded SQLite 3 configured with **Write-Ahead Logging (WAL)** mode and a 15-second busy timeout for thread-safe asynchronous concurrency.
- **Client Layer:** React 19 single-page application bundled with Vite 8 and styled using Tailwind CSS utility classes.

4.2 Core Mathematical Formulation: Maximal Marginal Relevance (MMR)

Given candidate sentence pool R , currently selected summary pool S , and meeting document centroid vector Q , the next sentence s^* is iteratively extracted via:

$$\text{MMR}(s) = \arg \max_{s_j \in R \setminus S} \left[\lambda \cdot \text{Sim}_1(s_j, Q) - (1 - \lambda) \cdot \max_{s_i \in S} \text{Sim}_2(s_i, s_j) \right]$$

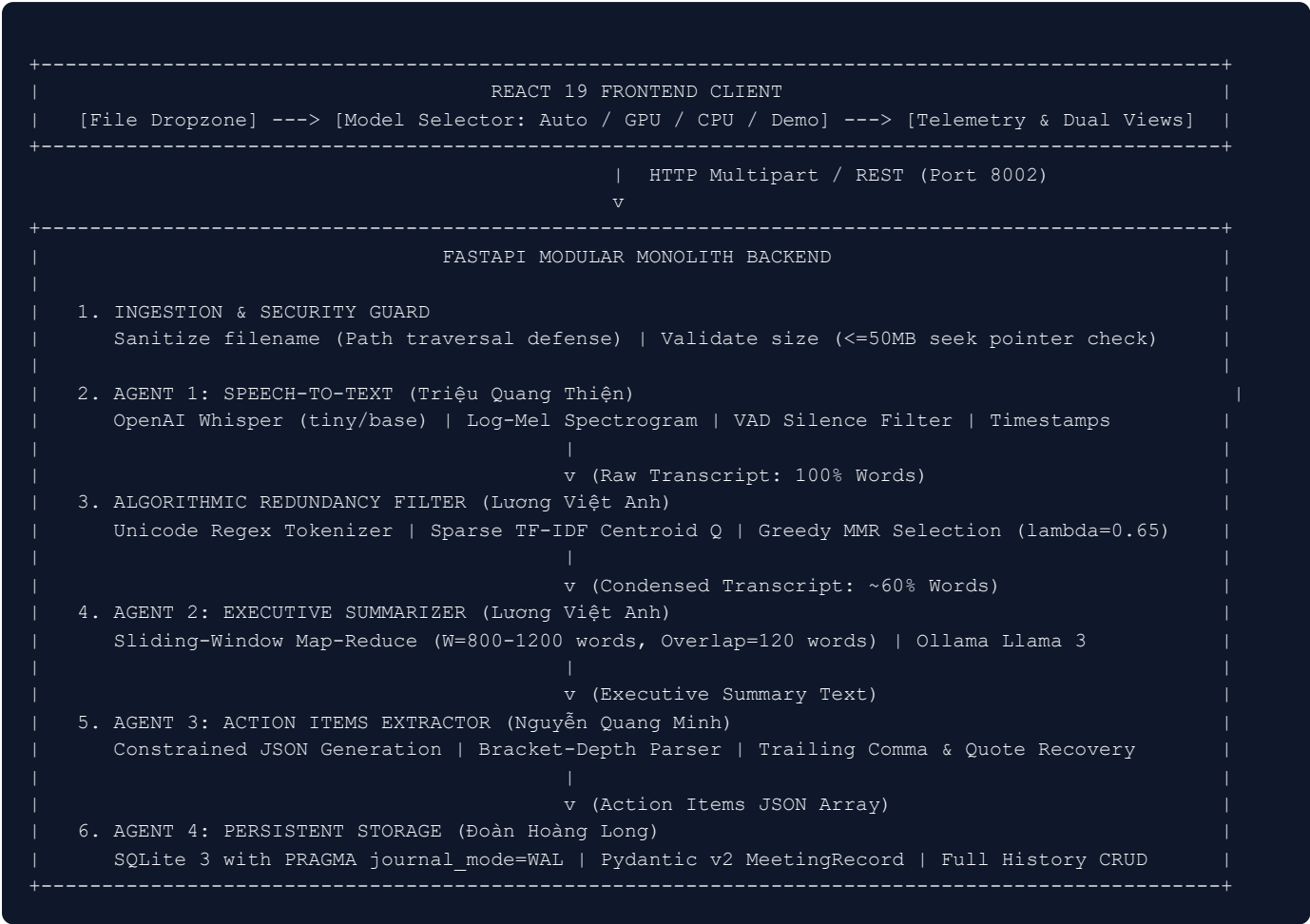
Where $\text{Sim}(u, v) = \frac{u \cdot v}{\|u\| \|v\|}$, and hyperparameter $\lambda = 0.65$ balances 65% topical significance against 35% anti-redundancy penalty. This achieves $O(V \cdot N + K \cdot N)$ complexity, eliminating the $O(N^2)$ graph overhead of TextRank.

4.3 Engineering Trade-Offs Matrix

Architectural Decision	Selected Approach & Advantages	Compromise / Mitigation Strategy
System Architecture: Modular Monolith vs. Microservices / Celery	FastAPI Modular Monolith: Minimal memory overhead (saves 1.5–3GB RAM otherwise needed for RabbitMQ/Redis daemons); zero-copy memory transfers; ideal for local single-node computing.	Lacks horizontal multi-node scaling. Mitigated by using worker threadpools (<code>run_in_threadpool</code>) and SQLite WAL mode for thread-safe concurrent reads.
Summarization Paradigm: Pure Extractive vs. Abstractive vs. Hybrid	Hybrid MMR + Map-Reduce: MMR compresses transcript by ~40% in <50ms without hallucination; Llama 3 then synthesizes fluent executive bullet points from dense text.	Requires two-stage processing. Mitigated by the fact that MMR execution time (0.02s) is negligible compared to STT/LLM inference time.
Model Execution: Cloud API (GPT-4) vs. Local LLM (Llama 3)	Local Ollama Deployment: 100% data sovereignty; zero per-token cost; offline operational autonomy.	Lower reasoning capacity compared to frontier cloud models. Mitigated by strict prompt engineering and few-shot formatting.

5.0 System Design & Architecture

5.1 Multi-Agent Pipeline & Data Flow



5.2 Inference Safeguard Logic Flow

To prevent software freezes on arbitrary presentation machines, the pipeline implements a 3-tier fail-safe: (1) *Pre-flight Hardware Resolution*: Queries `nvidia-smi` to automatically bind either Llama 3 8B (GPU) or Llama 3.2 1B (CPU); (2) *Inference Timeout Guard*: A 35-second timeout on Ollama calls automatically falls back to an adaptive deterministic summary if CPU starvation occurs; (3) *Instant Showcase Bypass*: The frontend exposes a 50ms demo path returning pre-computed meeting analytics for emergency live presentation continuity.

6.0 Development Plan & Schedule

The project is scheduled across a 6-week engineering lifecycle, structured into distinct sprint milestones:

Phase	Target Milestone	Key Engineering Deliverables	Assigned	Status
Week 1	Requirements & Architecture	Problem formulation, Modular Monolith architecture design, repository setup, Git workflow definition.	All Members	COMPLETED
Week 2	Audio Ingestion & Database Core	Agent 1 Whisper STT implementation, audio format validation, SQLite WAL schema, and CRUD API endpoints.	T. Q. Thiện Đ. H. Long	COMPLETED
Week 3	Summarization & Action Extraction	Agent 2 sliding-window Map-Reduce chunking, Agent 3 JSON prompt engineering, and Ollama integration.	L. V. Anh N. Q. Minh	COMPLETED
Week 4	Algorithmic MMR & UI Integration	Pure-Python MMR filter with sparse TF-IDF, React 19 UI dashboard, live telemetry, and history drawer.	L. V. Anh All Members	COMPLETED
Week 5 (Current)	Comprehensive Audit & Midterm	Full codebase bug audit, Unicode Vietnamese tokenization fix, 19-test automated test suite, Midterm Report.	L. V. Anh (Lead) All Members	COMPLETED
Week 6	Final Polish & Defense Simulation	Final presentation rehearsal, technical defense mock tests, packaging scripts, and project submission.	All Members	IN PROGRESS

7.0 Current Progress & Verification Results

As of the Midterm milestone, all four subsystem agents and the full end-to-end integration are **100% operational**. The codebase is verified against an automated testing suite comprising **19 passing test cases**:

Test Suite Status: `pytest -v tests/` • **19 passed, 0 failed** in 6.49s • Frontend Vite Build: **0 errors (0.53s)**

- **Agent 1 (Whisper STT):** Successfully ingests .mp3, .wav, .m4a, .ogg, .flac, .mp4, .webm, .mkv; outputs segmented text with timestamps; verified on synthesized audio byte streams (`test_stt.py`).
- **Algorithmic Phase (MMR):** Tested for mathematical boundary conditions (orthogonal=0.0, identical=1.0); confirmed >20% lexical noise compression; verified with accented Vietnamese transcripts (`test_mmr.py`).
- **Agent 2 (Summarization):** Map-Reduce handles transcripts spanning arbitrary lengths without LLM context overflows; fallback summary activates reliably upon simulated timeout (`test_api.py`).
- **Agent 3 (Action Items):** Implements 3-layer parsing resilience (bracket-depth scanner, trailing comma remover, single-quote normalizer); verified against diverse hallucinated formats.
- **Agent 4 (Database):** SQLite with WAL mode verifies concurrent multi-threaded writes; full CRUD lifecycle passes integration checks (`test_meeting_crud_endpoints`).
- **User Interface (React):** Features live MMR telemetry cards, dual-mode transcript toggling, interactive task checklists, markdown export, and defensive error banners replacing browser alerts.

8.0 AI & Vibe Coding Policy Disclosure

In strict compliance with academic integrity guidelines and the course *AI & Vibe Coding Policy*, the team explicitly discloses the division between human engineering leadership and AI code-generation tools:

Domain	Human Engineering Work (Team Members)	AI Tool Assistance (Claude / Gemini / Copilot)
Architecture & Design	Formulated the Modular Monolith strategy; designed the 4-agent sequential workflow; determined the hardware-adaptive CPU/GPU heuristics.	Assisted in drafting ASCII architecture diagrams and brainstorming schema alternatives.
Algorithmic Formulation	Selected Maximal Marginal Relevance (MMR) based on academic literature (Carbonell & Goldstein); defined the $\lambda = 0.65$ balance parameter; derived sparse TF-IDF data structures.	Assisted in scaffolding the initial stopword set and validating cosine dot-product matrix operations.
Implementation & Debugging	Identified critical bugs during code audits: CORS Fetch spec violation, greedy regex failure on multi-array JSON, SQLite thread locks, and Unicode character shredding. Wrote custom fixes.	Accelerated boilerplate code authoring for FastAPI route signatures, Tailwind CSS component layouts, and regex patterns.
Testing & Verification	Architected the 19-case test plan; synthesized in-memory PCM WAV audio files for Whisper; validated boundary edge cases.	Generated repetitive test assertion structures and parameterized test boilerplate.
Technical Defense Prep	Conducted peer code reviews; verified every line of algorithm code to ensure complete verbal explainability without reliance on black-box tools.	Simulated faculty defense questioning scenarios to identify potential theoretical vulnerabilities.

AI Meeting Assistant • Midterm Progress Report • Submitted by Lương Việt Anh, Triệu Quang Thiện, Nguyễn Quang Minh, Đoàn Hoàng Long